Exercise 03: Simulation Semantics

0.1 Blocking and Nonblocking Assignment Race

Objective: *Observe how Verilog scheduling can create a race between concurrent processes.*

Task: *Run a small Verilog simulation, inspect its waveform, and discuss the observed behaviour.*

Locate the exercise in

`lecture02/exercises/E03_simulation_semantics`.

The module contains two processes triggered by the same rising clock edge. One process changes `value` using a blocking assignment, while the other samples `value` using a nonblocking assignment. The order in which the two processes execute in the active simulation region is not defined by Verilog.

Run the simulation and open the generated FST waveform:

```
[<username>@<servername> E03_simulation_semantics]$ make 01_race_demo
[<username>@<servername> E03_simulation_semantics]$ gtkwave 01_race_demo.fst

# Clean files generated by this exercise
[<username>@<servername> E03_simulation_semantics]$ make clean_01_race_demo
```

Add `clk`, `value`, and `sampled` to the waveform view. Zoom in on the rising clock edges and discuss the following questions:

1. Does `sampled` receive the old or the new value of `value`?
2. Which of the two clocked processes appears to execute first?
3. Is the observed result guaranteed to be the same with another Verilog simulator?
4. Why might repeated runs with the same simulator still produce the same waveform even though the code contains a race?
5. How could the module be rewritten to remove the race?

The purpose of the exercise is observation and discussion. Do not modify the module before discussing the waveform.

0.2 Registered Status Feedback into Testbench Stimulus

Objective: *Investigate a race between two testbench processes that share an overflow counter.*

Task: *Run the supplied testbench, inspect its waveform, and explain why the counter is not always incorporated into the stimulus in the same way.*

Open `02_feedback_race.v` in

`exercises/E03_simulation_semantics`.

The file contains a small DUT and its top-level testbench. On each rising clock edge, the DUT adds two 8-bit inputs and registers the result in a 9-bit output. It also registers `ninth_bit_set` from bit 8—the ninth bit—of the calculated result. Both DUT outputs use nonblocking assignments. An active-high reset sets both outputs to zero, so the DUT itself has well-defined behaviour.

The testbench has two concurrent processes triggered by the same rising edge. The observer increments the shared `overflow_count` with a blocking assignment whenever it samples `ninth_bit_set` high. The stimulus process reads that counter and schedules inputs which produce `BASE_A + (BASE_B + overflow_count)`. Its input assignments are nonblocking, but the counter value used on their right-hand sides is evaluated immediately. The result therefore depends on whether the observer increments the counter before or after the stimulus process reads it.

Build the second example, then open its waveform:

```
[<username>@<servername> E03_simulation_semantics]$ make 02_feedback_race
[<username>@<servername> E03_simulation_semantics]$ gtkwave 02_feedback_race.fst

# Clean files generated by this exercise
[<username>@<servername> E03_simulation_semantics]$ make clean_02_feedback_race
```

Add `clk`, `reset`, `input_a`, `input_b`, `sum`, `ninth_bit_set`, `overflow_count`, and `debug_expected_sum` to the waveform. The race-free `debug_expected_sum` is calculated in the counter process after the optional increment. Compare its value at one rising edge with `sum` at the following rising edge. A difference shows that the stimulus process used the counter before it was incremented. Locate the first edge where `ninth_bit_set` is high, then discuss the following questions:

1. Are `ninth_bit_set` and `sum[8]` updated consistently by the DUT?
2. When `ninth_bit_set` is asserted, does `input_b` use the counter value from before or after the increment?
3. Which testbench process executes first, and is that order guaranteed?
4. Why do the nonblocking input assignments not remove the race involving the counter?
5. When does `sum` differ from the preceding value of `debug_expected_sum`, and what does this reveal?
6. How could the testbench be rewritten so that sampling and stimulus generation occur in a defined order?

As in the first exercise, observe and discuss the supplied implementation before changing it.

0.3 Reset Release on a Clock Edge

Objective: *Observe the race created when a testbench changes a synchronous reset at exactly the same time as a clock edge.*

Task: *Run a counter simulation, inspect the reset-release edge, and determine whether the counter resets or increments.*

Open `03_reset_release_race.v` in

`exercises/E03_simulation_semantics`.

The DUT is a 4-bit counter with an active-high synchronous reset. The clock has a period of 10 ns and therefore rises at 5 ns, 15 ns, 25 ns, and so on. The testbench releases reset using a blocking assignment at exactly 15 ns. The DUT and the reset process can consequently execute in either order in the active simulation region.

Run the third example and open its waveform:

```
[<username>@<servername> E03_simulation_semantics]$ make 03_reset_release_race
[<username>@<servername> E03_simulation_semantics]$ gtkwave 03_reset_release_race.fst

# Clean files generated by this exercise
[<username>@<servername> E03_simulation_semantics]$ make clean_03_reset_release_race
```

Add `clk`, `reset`, and `count` to the waveform and zoom in on 15 ns. Discuss the following questions:

1. Does the counter reset or increment at 15 ns?
2. Which value of `reset` did the DUT sample at that edge?
3. Is the result guaranteed to be the same with another simulator?
4. Why might repeated runs with one simulator always produce the same waveform?
5. Would changing the reset assignment to a nonblocking assignment remove the race, and would that be the clearest testbench solution?
6. How can reset be released at an unambiguous time?

After discussing the supplied waveform, try moving the reset release to a falling clock edge and compare the result.